

Intermediate Code Generation and Local Optimization

by

Md. Shahriar Karim

Source:

1. CS143 course, Stanford University
2. ECE468, by Prof. Milind Kulkarni, Purdue University
3. Crafting a Compiler in C, by Fisher and LeBlanc

Three Address Code (TAC)

- Intermediate code generated using information obtained from the Syntax analyzer.

↓ from Syntax Analyzer
Abstract Syntax Tree (AST)

↓
often used to generate TAC that is subsequently optimized in alternative approaches.

- There are high-level Assembly instructions where each operation has at most three operands.

However, instruction can have fewer operands as well.

Example: $t1 = t2 + t3$

Diagram illustrating the components of the instruction $t1 = t2 + t3$:

- $t1$ is labeled as **operand 3** (result location).
- $t2$ is labeled as **operand 1**.
- $+$ is labeled as **binary operator**.
- $t3$ is labeled as **operand 2**.

Here, two operands are with the binary operator and third operand $t1$ is the result location.

Interestingly, operand can compare to zero or to a second location to branch to.

Consider the arithmetic expression

$$a = b * c + b * d;$$

$$\begin{aligned} -t1 &= b * c; \\ -t2 &= b * d; \\ -t3 &= -t1 + -t2; \\ a &= -t3; \end{aligned}$$

Here $-t1, -t2, -t3$ are temp variables created by the compiler in order to keep the number of operands down to three.

Another example:

$$X = (-b + \text{sqrt}(b^2 - 4 * a * c)) / (2 * a)$$

The TAC instructions are

$$-t1 = b * b$$

$$-t2 = 4 * a$$

$$-t3 = t2 * c \quad \text{realizes } 4 * a * c$$

$$-t4 = -t1 - -t3$$

$$-t5 = \text{sqrt}(-t4)$$

$$-t6 = 0 - b \quad \text{for } -b$$

$$-t7 = -t5 + -t6$$

$$-t8 = 2 * a$$

$$-t9 = -t7 / -t8$$

$$X = t9$$

TAC is intermediate language within compiler.

- may have symbolic addresses, which are translated during register allocation.
- operands may not be concrete memory addresses or processor registers.

Another simple TAC expression

int a;		TAC
int b;		-to = b + c;
int c;		a = -to + d;
int d;		-t1 = a * a;
a = b + c + d;		-t2 = b * b;
b = a * a + b * b;		b = -t1 + -t2;

Concepts of Temporary Variables:

- The "Three" in TAC refers to the number of maximum operands in any instruction
- So, if an evaluation of any expression requires to deal with more than three subexpression, we need to introduce temporary variables.

int a;		TAC
int b;		-to = 5;
a = 5 + 2 * b		-t1 = 2 * b;
		a = -to + -t1

- TAC allows instructions with two operands as well.

▣ Variable assignment in TAC

$\text{var} = \text{constant};$

$\text{var1} = \text{var2};$

$\text{var1} = \text{var2} \text{ op } \text{var3};$

$\text{var1} = \text{constant} \text{ op } \text{var2};$

$\text{var1} = \text{var2} \text{ op } \text{constant};$

$\text{var} = \text{constant1} \text{ op } \text{constant2};$

Here, permitted operators are

$+, -, *, /, \%$

Q. How to compile $y = -x;$?

Ans: could be done in two ways

$y = 0 - x; \quad y = -1 * x;$

▣ TAC with Booleans

$\text{int } x;$

$\text{int } y;$

$\text{bool } b1;$

$\text{bool } b2;$

$\text{bool } b3;$

$b1 = x + x < y$

$b2 = x + y == y$

$b3 = x + x > y$

TAC

$-t0 = x + x$

$-t1 = y;$

$b1 = -t0 < -t1;$

$-t2 = x + x;$

$-t3 = y;$

$b2 = -t2 == -t3$

$-t4 = x + x;$

$-t5 = y;$

$b3 = -t5 < -t4$

More on TACs ...

- Boolean variables are represented as integers that have zero or non-zero values
- Besides arithmetic operators, TAC also supports $<$, $==$, $||$, and $!!$

Q. How can we compile $b = (x \leq y)$?

TAC : $-t_0 = x < y;$
 $-t_1 = x == y;$] could be splitted into multiple TAC instructions
 $b = -t_0 || -t_1$

Consider the below code [Control Flow Statements]

```
int x;
int y;
int z;
```

```
if (x < y)
```

```
    z = x;
```

```
else
```

```
    z = y;
```

```
z = z * z
```

```
-t0 = x < y;
```

```
ISZ -t0 Goto -L0;
```

```
    z = x;
```

```
Goto -L1;
```

```
-t0 = x < y;
```

```
-L0:
```

```
-L0:
```

```
    z = y;
```

```
-L1:
```

```
    z = z * z;
```

→ Label

→ Control flow

→ Control flow

* Examples are from Decaf programming language

So,

- TAC allows named labels, as used in previous example (-L0, -L1), to indicate particular portion of the code, which later be accessed using a jump process.
- TAC provides two control flow instructions
 - Goto label;
 - IfZ value Goto Label;

Where, IfZ is always paired with Goto

Another example:

```
int x;  
int y;  
while (x < y) {  
    x = x * 2;  
}  
y = x;
```

→ While loop

As we see, it works as iterative process

```
-L0: use the label to assist iteration  
    -t0 = x < y  
    IfZ -t0 Goto -L1  
    x = x * 2;  
    Goto -L0  
-L1:  
    y = x;
```

Consider C-like iterative loop $-t1 = i \times 10$

```

...
for (i=0; i<10; i=i+1) {
    b[i] = i*i;
}
...

```

* simple 'if' loop with goto in C

if $i \leq 10$ goto label

```

t1 = 0;    IfZ -t1 Goto L2
L1:  ifZ -t1 goto L2.
      conditional jump for i
t2 = t1 * t1    squaring t1
t3 = t1 * 4    word align address
t4 = b + t3    Address for t1
                t1 * t1, storing
*t4 = t2.
t1 = t1 + 1
goto L1
L2:  ...

```

More examples using Decaf programming language.

Design Proposal
Implemented as
part of implementation

→ Designed using LLVM
 → object oriented
 → has syntactic similarities to C, Java and Python-like implementations

Syntax related to Java.

```

while : while (x < 5) {
        y = y * 2;
        x = x + 1;
      }

```

```

for:   for (i=0; i<10; i=i+1)
      {
        y = y * 2;
      }

```

TAC example : a complete program

```
void main ( ) {  
    int x, y;  
    int i = x * x + y * y;  
    while (i > 5) {  
        i = i - x;  
    }  
}
```

TAC

```
main;  
    BeginFunc 24;  
    -t0 = x * x;  
    -t1 = y * y;  
    i = -t0 + -t1;  
-L0:  
    -t2 = 5 < i;  
    IfZ -t2 Goto -L1;  
    i = i - x;  
    Goto -L0;  
-L1:  
    EndFunc
```

Here, 4 steps —

- A label identifies the start of the function
- A "BeginFunc N": where N is the byte spaces for local and temporary variables.

- The body of the function

- An EndFunc: Instruction marking the end of the func

→ if end is reached, stack frames are ~~reached~~ cleared, and returns. ~~value~~

Reference: ① stanford university, CS143 course
② compilers, principles.... book by Ravi Sethi

Three-Address-Code (TAC) to Assembly

Often this transformation introduces assembly code that uses resources more than necessary to perform the desired task. For instance, consider the below code:

ADD A, B, C $\rightarrow$

1. LD A, R1 Load A to R1
2. LD B, R2 Load B to R2
3. ADD R1, R2, R3 Add content from R1 and R2
4. ST R3, C Store content from R3 to C

ADD C, A, E $\rightarrow$

5. LD C, R4
6. LD A, R5
7. ADD R4, R5, R6
8. ST R6, E

However, it is inefficient

$\rightarrow$ Redundant use of C through LD C command line 5
 $\rightarrow$ Redundant use of A through LD A in line 6
 $\rightarrow$ Overall, use a lot of Registers

Another example of inefficiency

ADD A, B, C $\rightarrow$ Same as line 1 to line 4 above

ADD A, B, D $\rightarrow$

- 5'. LD A, R4
- 6'. LD B, R5
- 7'. ADD R4, R5, R6
- 8'. ST R6, D

Here, A+B is already computed in line 3 and is recomputed in line 7'

So, we are wasting instructions, causing extra-time for code execution

So, such inefficiency in generated assembly code must be addressed and is done generally after the assembly generation. A few approaches used to improve performance are

- Address mode selection : Helps to choose better instructions.
- Peephole optimizations : Remove redundant expressions.
- Common subexpression elimination: $\rightarrow$ Addition of $A+B$ as in previous example.
- Register allocation:

 Reduces the number of registers used in the implementation

 removes redundant computation

Address Mode Selection (AMS)

- Selects optimal addressing mode ~~for~~ to access the registers
- Performed optimization of addressing mode can have
 - $\rightarrow$ code size
 - $\rightarrow$ execution timeas the performance objectives.

Simple instruction:

$\text{+ } A \quad B \quad C$

 can be indirect, register, memory address etc.

 literal, register, memory address etc.

 literal, register, memory address etc.

Peephole Optimization: Different Phases

- It is a process to increase the efficiency of the generated code by a compiler. Nearly all compilers do perform such optimization steps.
- This technique is done at different levels—generally it is done after ~~code~~ intermediate code generation. Interestingly, the peephole optimization can be performed on the AST level as well.
- Overall peephole optimization can be done in three different stages:
 - Can be performed on AST
 - On the Intermediate Code (IR) trees
 - On the Generated code

Generally, peephole optimization looks for patterns and replaces the pattern.

Pattern match $\xrightarrow{\text{Then}}$ Replacement

That is, when an instruction sequence or tree matching the "decided pattern" is found, the replacement is done.

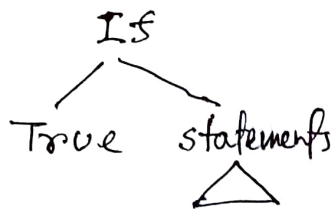
AST

Interestingly, if no patterns applied, then the code replacement does not occur and the code sequence is left unchanged.

Also, number of cases to be replaced could be unlimited.

Levels of Peephole Optimization

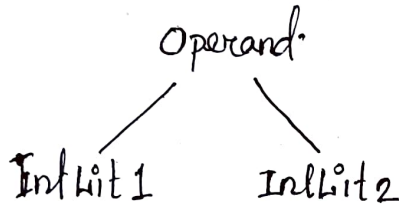
AST-level Optimization:



$\Rightarrow$



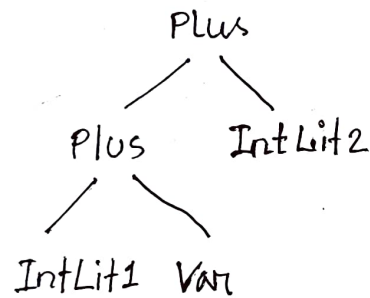
Condition of If when is always true, we don't need the condition part anymore



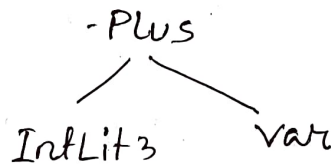
$\Rightarrow$

IntLit3

Here, operand is constant so, it is replaced with the value of expression. This is also known as constant folding.



$\Rightarrow$



As we have two Plus as the source and leaf, we can push the leaf "plus" to the root "plus" and the known $\text{IntLit1} + \text{IntLit2}$ are replaced by IntLit3

Implementation of AST-optimization

- Generally, tree rewriting tools, for instance BURS, can be used.
- Trees can be rewritten several times. That's, tree version already optimized is again considered for further optimization.

Peephole Optimizations : Why do we need it ?

- Remove redundancy at different levels
 - For instance, redundant instructions
- Generally, it is performed by pattern matching
- The process can recognize sequence of instructions that are possible to implement using single-line instruction, or by a reduced form.

For instance,

Peephole optimization approach forms a sliding window to focus on a small portion of the generated code or other forms.

and upon
Pattern matching.

Replace the code segment by a more efficient one.

- Replaced version is later scanned again along with the rest of the code for further refinement.

For instance,

ST R X
LD X R] Peephole removes LD part.

LDI R1 R2
ADD R1 4 R1] Can be replaced by single instruction
LDINC R1 R2 4

load immediate
could be constant
Sum to be placed

Strategies for Peephole optimizations

Constant folding:

MOV lit1, Rx		MOV lit1 + lit2, Ry
ADD lit2, Rx, Ry		

ADD lit1, lit2, Rx		MOV lit1 + lit2, Rx

So, by definition, it eliminates expression of the value calculation that can be known even before the execution of the code. For instance, the code segment

$i \leftarrow 10 * 20 * 32$
compiler does the calculation beforehand
to avoid runtime multiplication.

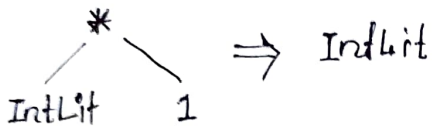
Null sequences:

MUL operand, 1, Rx		MOV operand, Rx
ADD operand, 0, Rx		

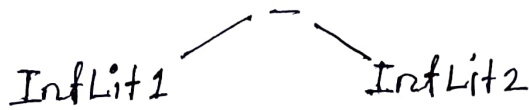
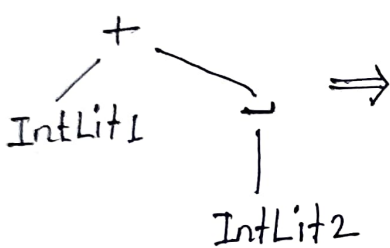
IR-level Peephole Optimization

Constant folding:

- evaluating the constant expressions at the compile time instead of leaving it for run-time calculation



Here, multiplication with 1 does not change IntLit, so, we apply constant folding.



Here, addition of a negative value is transformed to subtraction.

so
constant folding

Constant propagation:

Process to substitute known constant values in any expression during compile time

For instance,

```
int x = 14;
int y = 7 - x/2;
return y * (x + 2)
```

```
int x = 14;
int y = 7 - 14/2
return y * (196 + 2)
```

Another code (source: wiki)

```
int a = 30;
int b = 9 - (a/5);
int c;
c = b * 4
if (c > 10) {
    c = c - 10;
}
return c * (60/a)
```

```
int a = 30;
int b = 3;
int c;
if (c > 10) {
    c = c - 10;
}
return c * (2)
```

Here, constant propagation & constant folding both.